

Итерация №4

Интерфейсы LTI для judge0

Заказчик: Иванов Д.В.

Исполнители:	Гирман А.В.	1304
	Святохо В.В.	3344
	Тукалкин В.А.	3344
	Щербак М.С.	3344
	Пачев Д.К.	3344
	Охрименко Д.И.	3344

План на текущую итерацию

План текущей итерации:

- Написать тесты для проверки вариантов использования проекта.
- Подготовить полную инструкцию по эксплуатации проекта.
- Подготовить задача для возможных будущих дополнений к проекту.

Функциональные тесты

test_check.py – тесты эндпоинта /check:

- Проверки на правильный и неправильный Python код.
- Проверки на несуществующую задачу и недопустимый ЯП.
- Проверка на синтаксические ошибки.
- Проверки кода на Java и JavaScript.
- Проверка полного цикла использования (неправильный код меняется на правильный).

Функциональные тесты

`test_run.py` – тесты эндпоинта `/run`:

- Проверка запуска Python кода (обычного, с `stdin`, с ошибкой).
- Проверка запуска кода на Java и JavaScript.
- Проверка запуска с невалидным ЯП.
- Проверка полного цикла использования (последовательный запуск кода на разных языках).

Интеграционные тесты

test_auth.py – тесты аутентификации и авторизации:

- Проверка на вход (успешный вход, при неправильном пароле, при несуществующем пользователе).
- Проверка на токен (обновление токена, отсутствие токена, невалидный токен).
- Проверка на выход (успешный выход, выход без токена).
- Проверка сессии (получение новой сессии, сессия без токена).

Интеграционные тесты

`test_users.py` – тесты управления пользователями:

- Проверка на получении информации о пользователях (получение конкретного, несуществующего и списка всех пользователей, фильтрация по имени/логину, включение/исключение удаленных пользователей).
- Проверка на обновление информации о пользователях (полное и частичное обновление информации).
- Проверка удаления пользователя.
- Проверка полного цикла использования (создание, обновление и удаление).

Интеграционные тесты

`test_tasks.py` – тесты управления задачами:

- Проверка на получении задач (получение пустого списка, списка всех задач, фильтрация по названию и таймауту).
- Проверка на создание задачи (с тестами, при невалидном ЯП).
- Проверка на обновлении задачи (обновление полей, изменение ЯП).
- Проверка на удаление задачи.
- Проверка на получение тестов для задачи (получение пустого списка, списка всех тестов).
- Проверка на создание тестов.

Интеграционные тесты

- Проверка на обновление теста.
- Проверка на удаление теста.
- Проверка на импорта теста из JSON и обработка его ошибок.
- Проверка полного цикла использования (создание задачи, добавление тестов, обновление и удаление).

По аналогии написаны `test_analytics.py` (для аналитики отслеживания попыток) и `test_modules.py` (для управления модулями и задачами в них)

Требования к ресурсам

Вся система построена вокруг масштабируемой архитектуры, где количество рабочих процессов Judge0 регулируется в зависимости от пика одновременных проверок.

На нагрузку влияет:

- Пик проверок – максимальное количество студентов, одновременно нажимающих “Проверить”
- Время проверки – ~1.5 сек на один набор из 5 тестов
- Количество рабочих – масштабируется с нагрузкой (1 рабочий = 1 проверка одновременно)

Требования к ресурсам

Кол-во Студентов	Пик проверок	CPU (ядер)	RAM (ГБ)	SSD (ГБ)	Сеть (Мбит/с)
1	1	1	2	20	10
10	5	3	4	50	50
100	20	7	8	200	200
500	100	24	32	700	500

Сводная таблица минимальных ресурсов

Детальная информация по ресурсам: <https://github.com/moevm/mse1h2026-judge0-lti/wiki>

Результаты текущей итерации

В ходе работы над текущей итерацией, было сделано:

- Разработаны тесты проверяющие сценарии использования всего проекта.
- Подготовлена финализированная инструкция по запуску проекта.
- Проведены расчетные требования к ресурсам для эксплуатации проекта.

Задачи на будущее

- Реализовать возможность вернуть баллы за решение на мудл из IDE.
- Ограничить полномочия роли Преподаватель, добавить модель Курса, которая объединяет Модули, привязывать курс к преподавателю.